BỘ CÔNG THƯƠNG
TRƯỜNG ĐẠI HỌC CÔNG THƯƠNG TP. HCM
KHOA CÔNG NGHỆ THÔNG TIN

ĐỒ ÁN DỮ LIỆU LỚN

< HỆ THỐNG PHÁT HIỆN GIAN LẬN THẺ TÍN DỤNG REAL TIME>

Ngành: <Công nghệ thông tin>

SINH VIÊN THỰC HIỆN:

1. 2001230641_Nguyễn Thành Phát
2. 2033230060_Trần Hưu Đạt
3. 2001230669_Lê Thanh Phong

TP. HỒ CHÍ MINH, tháng 10 năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan rằng toàn bộ nội dung được trình bày trong báo cáo đồ án 'Xây dựng Hệ thống Phát hiện Gian lận Thẻ Tín dụng Real Time trên Nền tảng Big Data Lakehouse' là công trình nghiên cứu, thiết kế và hiện thực hóa thực tế của tập thể nhóm 15 dưới sự định hướng chuyên môn của Giảng viên hướng dẫn.

Các số liệu kiểm thử thực nghiệm, bảng đối sánh tham số thuật toán học máy (Precision@200, PR-AUC, ROC-AUC), kết quả đo lường độ trễ xử lý (P50, P95, P99), cũng như các mã nguồn xử lý luồng Spark Streaming, cấu trúc bảng Medallion Delta Lake, các pipeline điều phối Apache Airflow và giao diện Web điều hành đều được triển khai, kiểm thử thực tế 100% trên hệ thống máy chủ phát triển của nhóm tại phòng thực hành. Mọi tài liệu tham khảo, thư viện mã nguồn mở và bộ dữ liệu nghiên cứu kế thừa đều được trích dẫn nguồn gốc rõ ràng, minh bạch theo đúng chuẩn mực liêm chính học thuật.

Tôi xin chịu hoàn toàn trách nhiệm trước Hội đồng Khoa học và Ban Giám hiệu Nhà trường nếu có bất kỳ sự sao chép không trung thực nào bị phát hiện.

LỜI CẢM ƠN

Để hoàn thành đồ án tốt nghiệp này một cách trọn vẹn và đạt được những kết quả thực nghiệm sâu sắc, nhóm chúng em xin gửi lời cảm ơn chân thành và sâu sắc nhất đến toàn thể Quý Thầy Cô thuộc Khoa Công nghệ Thông tin - Trường Đại học Công Thương TP. Hồ Chí Minh. Trong suốt quá trình học tập tại trường, Quý Thầy Cô đã trang bị cho chúng em nền tảng kiến thức vững chắc về cấu trúc dữ liệu, hệ thống phân tán, cơ sở dữ liệu lớn và nguyên lý học máy hiện đại.

Đặc biệt, nhóm xin bày tỏ lòng biết ơn sâu sắc đến Giảng viên hướng dẫn đồ án. Thầy đã dành nhiều thời gian quý báu để định hướng kiến trúc hệ thống phân tán, gợi mở các giải pháp xử lý kỹ thuật phức tạp (như cơ chế chống rò rỉ dữ liệu Point-in-Time Join, tối ưu hóa độ trễ vi lô Spark Streaming, và giám sát trôi dạt dữ liệu Concept Drift) và tận tình chỉ bảo, đóng góp ý kiến hoàn thiện tài liệu báo cáo này.

Nhóm cũng xin chân thành cảm ơn gia đình, người thân và bạn bè đã luôn động viên, chia sẻ và tạo mọi điều kiện thuận lợi nhất để chúng em tập trung cao độ hoàn thành tốt đề tài. Do kiến thức và thời gian thực hiện còn có những giới hạn nhất định, đồ án chắc chắn không tránh khỏi những thiếu sót. Chúng em rất mong nhận được những nhận xét, góp ý quý báu từ Quý Thầy Cô trong Hội đồng phản biện để hệ thống được hoàn thiện hơn nữa trong tương lai.

TÓM TẮT ĐỀ TÀI

Trong kỷ nguyên thanh toán số bùng nổ, gian lận thẻ tín dụng trở thành vấn nạn đe dọa trực tiếp đến an toàn tài sản của khách hàng và uy tín của các định chế tài chính. Các hệ thống phát hiện gian lận truyền thống hoạt động dựa trên luật tĩnh (rule-based) hoặc xử lý mẻ (batch) bộc lộ sự chậm trễ nghiêm trọng và tỷ lệ cảnh báo sai (false positive) rất cao.

Đề tài này nghiên cứu, thiết kế và hiện thực hóa thành công một hệ sinh thái Big Data Lakehouse hoàn chỉnh phục vụ phát hiện gian lận và chấm điểm rủi ro thẻ tín dụng thời gian thực. Hệ thống tích hợp đồng bộ các công nghệ tiên tiến nhất bao gồm: Apache Kafka (hàng đợi thông điệp phân tán), Apache Spark Structured Streaming (động cơ xử lý vi lô với Watermark 1 phút và Stream-Static Join), Delta Lake 3.2 (hồ lưu trữ dữ liệu tuân thủ chuẩn Medallion Bronze - Silver - Gold, ACID, CDF, Time Travel), Spark MLlib (mô hình phân loại GBTClassifier tối ưu hóa Precision@200 và PR-AUC kết hợp Point-in-Time Join chống rò rỉ dữ liệu), Apache Airflow (điều phối 4 DAGs tự động hóa giám sát, phát hiện Drift và tái huấn luyện) và Giao diện Web Flask Dashboard điều hành trực tiếp tích hợp bàn làm việc thẩm định Human-In-The-Loop.

Kết quả nghiệm thu thực tế với chế độ tải cao Turbo Mode (300 giao dịch/giây) chứng minh hệ thống đạt độ trễ trung vị P50 = 480ms – 820ms, P95 = 1.285ms (đáp ứng xuất sắc cam kết SLA < 5s); mô hình đạt Precision@200 = 0.4800 (vượt xa chỉ tiêu cam kết >= 0.35) và PR-AUC = 0.8814 (vượt xa chỉ tiêu >= 0.75), vượt qua 100% 8 tiêu chuẩn chấp nhận hệ thống cấp ngân hàng.

Từ khóa: Big Data Lakehouse, Delta Lake, Spark Structured Streaming, Apache Kafka, Apache Airflow, Point-in-Time Join, Concept Drift (PSI, KS-test), Precision@200, Human-In-The-Loop.

MỤC LỤC

LỜI CAM ĐOAN 1
LỜI CẢM ƠN 3
TÓM TẮT ĐỀ TÀI 4
MỤC LỤC 5
DANH MỤC CÁC TỪ VIẾT TẮT 7
DANH MỤC CÁC BẢNG TRONG BÁO CÁO 8
CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ BÀI TOÁN GIAN LẬN THẺ TÍN DỤNG 9
1.1. Bối cảnh chuyển đổi số và tính cấp thiết của đề tài 9
1.2. Mục tiêu nghiên cứu và phạm vi đề tài 9
1.3. Những thách thức kỹ thuật cốt lõi trong dữ liệu ngân hàng 10
1.4. Bố cục của báo cáo khoa học 10
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC CÔNG NGHỆ NỀN TẢNG 12
2.1. Kiến trúc phân tầng dữ liệu Medallion Architecture 12
2.2. Hồ lưu trữ dữ liệu mở Delta Lake 3.2 và các tính năng mở rộng 12
2.3. Hàng đợi phân tán Apache Kafka & Spark Structured Streaming 13
2.4. Lý thuyết phân loại mất cân bằng và giám sát trôi dạt dữ liệu 13
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG 15
3.1. Kiến trúc tổng thể hệ thống (System Architecture) 15
3.2. Thiết kế lược đồ dữ liệu Delta Lake 15
3.3. Thiết kế cơ chế chống rò rỉ dữ liệu (Point-in-Time Join) 16
3.4. Thiết kế cơ chế điều phối 4 cửa sổ Terminal và giao thức IPC đồng bộ 16
3.5. Thiết kế vòng lặp phản hồi thẩm định Human-In-The-Loop 17
CHƯƠNG 4: HIỆN THỰC HÓA VÀ TRIỂN KHAI HỆ THỐNG 18
4.1. Môi trường triển khai và hạ tầng cụm phân tán 18
4.2. Hiện thực hóa phân hệ Streaming chấm điểm thời gian thực (Cửa sổ 2) 18
4.3. Hiện thực hóa phân hệ Stateful Streaming cập nhật trạng thái (Cửa sổ 3) 19
4.4. Hiện thực hóa bộ phát dữ liệu đa chế độ và đồng bộ Web (Cửa sổ 4) 19
4.5. Hiện thực hóa giao diện Web điều hành và bàn làm việc thẩm định (Cửa sổ 1) 19
4.6. Hiện thực hóa các Pipeline điều phối Apache Airflow 19
CHƯƠNG 5: THỬ NGHIỆM, ĐÁNH GIÁ VÀ NGHIỆM THU HỆ THỐNG 21
5.1. Kịch bản thử nghiệm áp lực cao (Turbo Mode Stress Test) 21

5.2. Đánh giá đối sánh hiệu năng các mô hình học máy 21
5.3. Đánh giá độ trễ xử lý đầu cuối (End-to-End Latency) 22
5.4. Kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống 22
5.5. Phân tích các ca xử lý lỗi kỹ thuật thực tế trong quá trình phát triển 23
CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 24
6.1. Các kết quả và đóng góp chính của đề tài 24
6.2. Hạn chế còn tồn tại 24
6.3. Định hướng phát triển trong tương lai 24
TÀI LIỆU THAM KHẢO 26
PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH HỆ THỐNG 27
Phụ lục A: Hướng dẫn cấu hình Docker Compose và kiểm tra hạ tầng 27
Phụ lục B: Hướng dẫn khởi động hệ thống 4 cửa sổ đồng bộ bằng chay.ps1 27
Phụ lục C: Hướng dẫn làm sạch dữ liệu và chạy bộ kiểm thử nghiệm thu 27

DANH MỤC CÁC TỪ VIẾT TẮT

Từ viết tắt	Giải thích ý nghĩa kỹ thuật
ACID	Atomicity, Consistency, Isolation, Durability (Thuộc tính toàn vẹn giao dịch)
CDF	Change Data Feed (Luồng dữ liệu thay đổi cấp hàng trên Delta Lake)
DAG	Directed Acyclic Graph (Đồ thị có hướng không chu trình trong Apache Airflow)
ETL	Extract, Transform, Load (Quy trình trích xuất, biến đổi và nạp dữ liệu)
GBT	Gradient Boosted Trees (Thuật toán cây quyết định tăng cường độ dốc)
HDFS	Hadoop Distributed File System (Hệ thống tập tin phân tán Hadoop)
IPC	Inter-Process Communication (Cơ chế giao tiếp liên tiến trình)
KS-Test	Kolmogorov-Smirnov Test (Kiểm định thống kê so sánh hai hàm phân phối thực nghiệm)
MLOps	Machine Learning Operations (Chu trình vận hành và triển khai học máy tự động)
PCA	Principal Component Analysis (Phương pháp phân tích thành phần chính)
PR-AUC	Precision-Recall Area Under Curve (Diện tích dưới đường cong Độ chính xác - Độ phủ)
PSI	Population Stability Index (Chỉ số ổn định phân phối dữ liệu phát hiện Drift)
ROC-AUC	Receiver Operating Characteristic - Area Under Curve
SLA	Service Level Agreement (Cam kết mức chất lượng dịch vụ kỹ thuật)

DANH MỤC CÁC BẢNG TRONG BÁO CÁO

Ký hiệu bảng	Tên bảng dữ liệu
Bảng 2.1	So sánh đặc tính kỹ thuật giữa Data Lake truyền thống và Delta Lakehouse
Bảng 3.1	Quy cách và cấu trúc 5 bảng dữ liệu Medallion Architecture trong Lakehouse
Bảng 4.1	Thông số phần cứng, cổng mạng và môi trường triển khai cụm phân tán
Bảng 5.1	Bảng đối sánh hiệu năng 3 thuật toán x 2 chiến lược mất cân bằng trên tập Test
Bảng 5.2	Đo lường độ trễ xử lý End-to-End tại các mốc phân vị (P50, P95, P99)
Bảng 5.3	Tổng hợp kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống (Acceptance Criteria)

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI VÀ BÀI TOÁN GIAN LẬN THẺ TÍN DỤNG

1.1. Bối cảnh chuyển đổi số và tính cấp thiết của đề tài

Trong bối cảnh bùng nổ của nền kinh tế số và làn sóng thanh toán không tiền mặt trên toàn cầu, các giao dịch thẻ tín dụng và thanh toán trực tuyến đã trở thành huyết mạch tài chính của hàng triệu cá nhân và doanh nghiệp. Tuy nhiên, song hành cùng sự thuận tiện vượt bậc là mối đe dọa thường trực từ tội phạm công nghệ cao với các thủ đoạn gian lận tinh vi, biến tướng khôn lường như đánh cắp thông tin thẻ (skimming), tấn công giả mạo (phishing), chiếm đoạt tài khoản (account takeover) và rửa tiền xuyên biên giới. Theo báo cáo thường niên của các tổ chức thẻ quốc tế (Visa, Mastercard), thiệt hại do gian lận thẻ tín dụng toàn cầu gây tổn thất hàng chục tỷ USD mỗi năm, làm suy giảm uy tín thương hiệu và gây tổn hại nghiêm trọng đến lòng tin của người tiêu dùng.

Trước đây, các tổ chức ngân hàng chủ yếu áp dụng phương thức phòng chống gian lận dựa trên tập luật tĩnh (rule-based system) hoặc đối soát mẻ (batch processing). Phương pháp này bộc lộ những lỗ hổng chí mạng: (1) Tính thụ động và độ trễ cao: Việc phát hiện giao dịch gian lận sau vài giờ hoặc cuối ngày đồng nghĩa với việc tiền đã bị tẩu tán khỏi hệ thống; (2) Tỷ lệ cảnh báo sai (False Positive) rất cao, dẫn đến việc khóa nhầm thẻ của khách hàng hợp lệ, gây ức chế và làm gián đoạn trải nghiệm giao dịch; (3) Khả năng thích ứng kém: Khi tội phạm thay đổi hành vi thanh toán, đội ngũ quản trị phải mất nhiều tuần để phân tích dữ liệu và cập nhật thủ công các bộ luật mới.

Do đó, việc nghiên cứu và xây dựng một hệ sinh thái hồ lưu trữ dữ liệu lớn (Lakehouse) có năng lực tiếp nhận dòng dữ liệu cực lớn, làm sạch tự động, trích xuất đặc trưng cửa sổ trượt hành vi và chấm điểm rủi ro bằng mô hình học máy trong thời gian thực (Real-time Fraud Detection) với độ trễ dưới 2 giây là bài toán có tính cấp thiết sống còn đối với sự an toàn tài chính số hiện nay.

1.2. Mục tiêu nghiên cứu và phạm vi đề tài

Mục tiêu tổng quát của đề tài là xây dựng một nền tảng Big Data Lakehouse hoàn chỉnh, ổn định và tự động hóa cao, phục vụ giám sát và phát hiện gian lận thẻ tín dụng thời gian thực. Cụ thể, các mục tiêu chi tiết bao gồm:

- 1. Xây dựng hạ tầng dữ liệu phân tán: Tích hợp cụm lưu trữ phân tán Hadoop HDFS 3.2.1 (gồm 1 NameNode và 3 DataNodes), hàng đợi thông điệp phân tán Apache Kafka 2.12 và động cơ xử lý dữ liệu vi mô Spark Structured Streaming.
- 2. Hiện thực hóa kiến trúc Medallion Lakehouse trên nền tảng Delta Lake 3.2: Thiết lập 5 bảng dữ liệu phân tầng nghiêm ngặt (bronze, silver, gold, quarantine) đảm bảo các tính chất ACID, kiểm soát phiên bản (Time Travel), lưu vết thay đổi cấp hàng (Change Data Feed) và ràng buộc toàn vẹn dữ liệu (CHECK constraints).
- 3. Tối ưu hóa mô hình Học máy chống gian lận: Huấn luyện và đóng gói mô hình học máy (GBTClassifier, Random Forest) trên không gian dữ liệu mất cân bằng cực đoan (tỷ

lệ gian lận 0.17%), áp dụng kỹ thuật Point-in-Time Join chống rò rỉ dữ liệu lịch sử và tối ưu hóa theo chỉ số thực tế Precision@200 và PR-AUC.

- 4. Tự động hóa vận hành MLOps bằng Apache Airflow: Xây dựng 4 luồng đồ thị DAGs tự động hóa giám sát dịch vụ streaming, kiểm tra trôi dạt dữ liệu Concept Drift (bằng chỉ số PSI và kiểm định Kolmogorov-Smirnov), kích hoạt tái huấn luyện mô hình khi có nhãn mới và bảo trì hồ dữ liệu (Z-ORDER, VACUUM).
- 5. Xây dựng ứng dụng Web điều hành và Bàn làm việc thẩm định (Human-In-The-Loop): Cung cấp giao diện trực quan cho chuyên viên ngân hàng tra cứu lịch sử thẻ, giám sát hàng đợi cảnh báo rủi ro thời gian thực và ghi nhận nhãn vàng thẩm định làm sạch mô hình.

1.3. Những thách thức kỹ thuật cốt lõi trong dữ liệu ngân hàng

Trong quá trình phát triển hệ thống dữ liệu lớn phục vụ lĩnh vực ngân hàng, đề tài đã trực tiếp giải quyết 4 thách thức kỹ thuật cốt lõi:

- 1. Thách thức về độ trễ cực thấp (Sub-second Latency): Các giao dịch thẻ tín dụng đòi hỏi quyết định phê duyệt hoặc cảnh báo rủi ro phải diễn ra trong vòng < 2 giây ngay khi quẹt thẻ, trong khi hệ thống phải đồng thời thực hiện tra cứu lịch sử chi tiêu của tài khoản trong các cửa sổ thời gian 1 giờ, 24 giờ, 7 ngày và 30 ngày.
- 2. Thách thức mất cân bằng dữ liệu cực đoan (Extreme Class Imbalance): Trong tập dữ liệu giao dịch thực tế ULB Machine Learning Group, chỉ có 492 giao dịch gian lận trên tổng số 284.807 giao dịch (tỷ lệ 0.172%). Các thuật toán phân loại tiêu chuẩn thường bị thiên lệch về lớp hợp lệ (majority class), đòi hỏi các kỹ thuật điều chỉnh hàm mất mát (WeightCol) hoặc Undersampling chuyên biệt.
- 3. Thách thức trôi dạt dữ liệu và hành vi (Concept & Data Drift): Thói quen chi tiêu của khách hàng biến động liên tục theo thời gian (dịp lễ Tết, khuyến mãi) và hành vi gian lận cũng liên tục thay đổi để né tránh kiểm soát. Hệ thống phải có năng lực đo lường độ trôi dạt thống kê khách quan và kích hoạt tái huấn luyện kịp thời.
- 4. Thách thức toàn vẹn giao dịch và rò rỉ dữ liệu (ACID & Data Leakage): Hệ thống vừa phải đảm bảo tính nhất quán dữ liệu khi hàng ngàn giao dịch ghi đồng thời vào hồ dữ liệu (ACID), vừa phải ngăn chặn tuyệt đối hiện tượng rò rỉ thông tin tương lai trong quá trình tạo tập dữ liệu huấn luyện (Training Feature Leakage).

1.4. Bố cục của báo cáo khoa học

Báo cáo khoa học này được tổ chức thành 6 chương nội dung chính, phản ánh tuần tự toàn bộ quá trình nghiên cứu, thiết kế, hiện thực hóa và đánh giá thực nghiệm của đề tài:

- Chương 1: Tổng quan về đề tài và bài toán gian lận thẻ tín dụng.
- Chương 2: Cơ sở lý thuyết và các công nghệ nền tảng (Medallion Lakehouse, Delta Lake, Kafka, Spark Streaming, Học máy mất cân bằng và Kiểm định Drift).
- Chương 3: Phân tích và thiết kế kiến trúc hệ thống (Thiết kế 5 tầng kiến trúc, lược đồ Delta Lake, Point-in-Time Join, cơ chế điều phối 4 cửa sổ và Human-In-The-Loop).

- Chương 4: Hiện thực hóa và triển khai hệ thống (Cấu hình cụm Docker HDFS, hiện thực hóa Cửa sổ 2 Chấm điểm, Cửa sổ 3 Cập nhật trạng thái, Cửa sổ 4 Phát dữ liệu, Web Flask Cửa sổ 1 và Airflow DAGs).
- Chương 5: Thử nghiệm, đánh giá và nghiệm thu hệ thống (Thử nghiệm Turbo Mode 300 tx/s, đánh giá đối sánh ML, đo lường độ trễ SLA, nghiệm thu 8 tiêu chuẩn chấp nhận và phân tích các ca xử lý lỗi kỹ thuật thực tế).
- Chương 6: Kết luận và hướng phát triển trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÁC CÔNG NGHỆ NỀN TẢNG

2.1. Kiến trúc phân tầng dữ liệu Medallion Architecture

Kiến trúc Medallion (hay còn gọi là mô hình phân tầng Bronze - Silver - Gold) là chuẩn mực thiết kế dữ liệu tiên tiến nhất được Databricks đề xuất nhằm đảm bảo chất lượng, cấu trúc và tính sẵn sàng của dữ liệu khi di chuyển qua các giai đoạn xử lý trong hồ lưu trữ dữ liệu (Lakehouse).

• Tầng Đồng (Bronze Layer - Raw Data): Là nơi tiếp nhận dữ liệu thô nguyên bản từ các nguồn phát sinh bên ngoài (như luồng Kafka, API đối tác, tệp nhật ký giao dịch). Dữ liệu được ghi nối tiếp (append-only) theo đúng định dạng ban đầu, có gắn dấu thời gian nạp và bảo toàn toàn vẹn lịch sử không qua chỉnh sửa.

• Tầng Bạc (Silver Layer - Cleansed & Conformed): Dữ liệu từ tầng Bronze được trích xuất, làm sạch, loại bỏ bản ghi trùng lặp, chuẩn hóa định dạng ngày tháng, kiểm tra tính hợp lệ của khóa chính và số tiền giao dịch. Các bản ghi vi phạm tính toàn vẹn sẽ được chuyển vào vùng cách ly (Quarantine). Đồng thời, tầng Silver duy trì các bảng hồ sơ trạng thái khách hàng được cập nhật liên tục (Upsert) đại diện cho 'sự thật duy nhất' (Single Source of Truth).

• Tầng Vàng (Gold Layer - Business Aggregates): Là tầng dữ liệu phục vụ trực tiếp cho người dùng nghiệp vụ, mô hình học máy và báo cáo phân tích cấp cao. Dữ liệu tại tầng Gold đã được chấm điểm rủi ro, xếp hạng phân vị, tính toán độ tin cậy và gắn kèm các yếu tố giải thích minh bạch (Explainable AI - XAI).

2.2. Hồ lưu trữ dữ liệu mở Delta Lake 3.2 và các tính năng mở rộng

Delta Lake 3.2 là lớp lưu trữ mã nguồn mở (Open Storage Layer) đột phá, mang các đặc tính toàn vẹn dữ liệu của kho dữ liệu truyền thống (Data Warehouse) đặt lên trên khả năng mở rộng quy mô không giới hạn của hồ dữ liệu (Data Lake). Để giải quyết bài toán tài chính, đề tài khai thác 5 tính năng cốt lõi:

1. Transaction Log (_delta_log): Mọi thao tác ghi, sửa, xóa đều được ghi nhận dưới dạng các file commit JSON tuần tự (00000000000000000000.json, ...) và checkpoint Parquet định kỳ, đảm bảo tính nguyên tử (Atomicity) và cô lập (Isolation) ở mức Serializable.

2. Thao tác MERGE INTO (Upsert): Cho phép cập nhật trạng thái giao dịch và lịch sử tài khoản khách hàng chỉ trong một lệnh thực thi duy nhất, đối chiếu qua khóa chính mà không cần sao chép bảng.

3. Du hành thời gian (Time Travel): Hỗ trợ truy vấn dữ liệu chính xác tại một phiên bản hoặc thời điểm bất kỳ trong quá khứ (`VERSION AS OF` hoặc `TIMESTAMP AS OF`), phục vụ công tác thanh tra kiểm toán và khôi phục sự cố.

4. Luồng dữ liệu thay đổi (Change Data Feed - CDF): Tự động tách biệt các dòng vừa được chèn mới (insert), cập nhật trước/sau (update_preimage/update_postimage) hoặc xóa (delete), cho phép các luồng xử lý phía sau tiêu thụ dữ liệu thay đổi cực kỳ hiệu quả.

5. Ràng buộc toàn vẹn CHECK Constraint: Thiết lập trực tiếp trên bảng Delta ở mức lưu trữ (ví dụ: `CHECK (so_tien > 0)`), lập tức từ chối và phát ngoại lệ nếu có bản ghi sai lệch lọt vào.

Bảng 2.1: So sánh đặc tính kỹ thuật giữa Data Lake truyền thống và Delta Lakehouse

Thuộc tính kỹ thuật	Data Lake truyền thống (Parquet/HDFS)	Delta Lakehouse (Đề tài áp dụng)
Giao dịch ACID	Không hỗ trợ (dễ rách file, đọc bẩn khi ghi đồng thời)	Hỗ trợ đầy đủ ACID thông qua commit log JSON tuần tự.
Cập nhật bản ghi (Update/Merge)	Không hỗ trợ (phải viết đè toàn bộ phân vùng)	Hỗ trợ lệnh MERGE INTO nguyên tử trên từng khóa chính.
Kiểm tra lược đồ (Schema)	Schema-on-read (dễ nạp dữ liệu sai cấu trúc)	Schema Enforcement & Schema Evolution tự động ngăn chặn lỗi.
Truy vấn lịch sử (Time Travel)	Phải tự tạo thư mục bản sao thủ công	Truy vấn trực tiếp qua cú pháp `VERSION AS OF` hoặc `TIMESTAMP AS OF`.
Luồng thay đổi dữ liệu (CDF)	Không có, phải quét toàn bộ bảng để đối soát	Change Data Feed ghi nhận chi tiết bản ghi trước và sau biến động.

2.3. Hàng đợi phân tán Apache Kafka & Spark Structured Streaming

Hệ thống sử dụng Apache Kafka Broker làm hàng đợi thông điệp chịu lỗi, tiếp nhận các gói tin giao dịch JSON từ máy trạm phát sinh. Apache Spark Structured Streaming đóng vai trò là động cơ xử lý trung tâm:

• Cơ chế Micro-Batch: Dữ liệu từ Kafka được gom thành các mẻ vi mô (micro-batches) với chu kỳ kích hoạt 2-3 giây, kết hợp xử lý song song trên nhiều lõi CPU.

• Chiến lược Watermark thời gian sự kiện: Để đối phó với hiện tượng mạng trễ làm 15% giao dịch đến lệch thứ tự lên tới 30 giây, hệ thống thiết lập Watermark thời gian sự kiện là 1 phút (`.withWatermark('thoi_diem', '1 minute')`). Spark sẽ duy trì vùng nhớ trạng thái (state store) để chờ đợi và sắp xếp đúng thứ tự các giao dịch phát sinh trước đó mà không bị loại bỏ oan uổng.

• Cơ chế Dual-Sink: Một điểm sáng tạo trong thiết kế là trong cùng một micro-batch, luồng dữ liệu được tách nhánh ghi đồng thời vào hai đích: (1) Ghi nối tiếp vào `bronze.giao_dich` lưu trữ bền vững; (2) Chấm điểm rủi ro qua mô hình học máy và ghi kết quả cảnh báo vào `gold.diem_rui_ro`.

2.4. Lý thuyết phân loại mất cân bằng và giám sát trôi dạt dữ liệu

Trong bài toán gian lận thẻ tín dụng, lớp thiểu số (gian lận, nhãn 1) chỉ chiếm 0.17%. Do đó, các độ đo truyền thống như Accuracy hay ROC-AUC bị sai lệch nghiêm trọng. Đề tài tập trung vào hai độ đo chuyên dụng:

• Precision@200: Độ chính xác tính trên Top 200 giao dịch có điểm số rủi ro dự báo cao nhất trong ngày:

Precision@200 = (Số gian lận thực tế trong Top 200) / 200

Chỉ số này phản ánh trực tiếp hiệu quả lao động thực tế của đội ngũ thẩm định viên phòng chống rủi ro ngân hàng (vốn chỉ có năng lực thẩm định tối đa 200 giao dịch mỗi ngày).

• PR-AUC (Precision-Recall Area Under Curve): Diện tích dưới đường cong liên hệ giữa độ chính xác (Precision) và độ phủ (Recall). PR-AUC phản ánh năng lực phân tách thực sự của mô hình trên tập dữ liệu mất cân bằng cực đoan mà không bị thổi phồng bởi lớp đa số hợp lệ.

Để phát hiện hiện tượng trôi dạt dữ liệu (Data Drift) giữa dữ liệu sản xuất và dữ liệu cơ sở (Baseline), hệ thống áp dụng hai chỉ số kiểm định thống kê:

$$PSI = \sum (A_b - E_b) \times ln(A_b / E_b)$$

Trong đó Ab là tỷ lệ giao dịch thực tế và Eb là tỷ lệ kỳ vọng tại phân vị b. Khi PSI > 0.25, phân phối dữ liệu bị trôi dạt nghiêm trọng và hệ thống sẽ tự động kích hoạt cảnh báo tái huấn luyện.

$$D = sup_x \mid F_1(x) - F_2(x) \mid$$

Khi giá trị kiểm định p-value < 0.01 và khoảng cách cực đại D > 0.15, hệ thống khẳng định phân phối các đặc trưng giao dịch đã bị dịch chuyển có ý nghĩa thống kê.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ KIẾN TRÚC HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống (System Architecture)

Hệ thống được thiết kế theo mô hình kiến trúc phân tán 5 tầng kết hợp, đảm bảo tính phân tách trách nhiệm (Separation of Concerns), khả năng mở rộng quy mô độc lập và độ tin cậy tuyệt đối:

- 1. Tầng Thu nhận & Phát sinh dữ liệu (Ingestion Layer): Bộ phát giao dịch trực tuyến (data/generator.py) mô phỏng dòng giao dịch thẻ quốc tế và nội địa, đẩy vào Kafka topic 'transactions' với tỷ lệ trễ ngẫu nhiên (out-of-order <= 30s).
- 2. Tầng Động cơ Xử lý Luồng (Streaming Engine): Gồm 2 tiến trình Spark Streaming độc lập: (a) Cửa sổ 2 (cham_diem.py) tiếp nhận luồng, Stream-Static Join với hồ sơ khách hàng, chấm điểm ML và Dual-Sink; (b) Cửa sổ 3 (cap_nhat_trang_thai.py) duy trì cửa sổ trượt hành vi khách hàng 1h, 24h, 7d, 30d và dọn dẹp tài khoản không hoạt động quá 30 ngày.
- 3. Tầng Lưu trữ Hồ dữ liệu (Lakehouse Storage Layer): 5 bảng Delta Lake phân tầng nghiêm ngặt (bronze, silver, gold, quarantine) lưu trữ trên cụm phân tán Hadoop HDFS (1 NameNode + 3 DataNodes) với hệ số nhân bản replication = 3.
- 4. Tầng Điều phối Tự động hóa MLOps (Orchestration Layer): Apache Airflow điều phối 4 DAGs tự động hóa giám sát streaming, kiểm tra drift thống kê, huấn luyện lại mô hình Point-in-Time Join và bảo trì Z-ORDER/VACUUM.
- 5. Tầng Ứng dụng & Bàn làm việc Thẩm định (Application & Reviewer Layer): Web Server Flask (web/app.py, port 5000) cung cấp giao diện dashboard điều hành luồng 8 bước, hàng đợi cảnh báo rủi ro thời gian thực, bàn làm việc thẩm định tương tác và giao diện kiểm toán Time Travel.

3.2. Thiết kế lược đồ dữ liệu Delta Lake

Bảng 3.1: Quy cách và cấu trúc 5 bảng dữ liệu Medallion Architecture trong Lakehouse

Tên bảng Delta	Tầng	Khóa / Phân vùng	Cơ chế ghi	Mục đích nghiệp vụ
bronze.giao_dich	Bronze	ma_giao_dich / Partition: ngay	Append-Only	Lưu vết toàn bộ giao dịch thô nguyên bản, áp dụng ràng buộc CHECK (so_tien > 0).
silver.trang_thai_khach_hang	Silver	tai_khoan (Primary Key)	Delta MERGE (Upsert)	Lưu trữ đặc trưng hành vi cửa sổ trượt (1h, 24h, 7d, 30d mean/stddev) của khách hàng.
silver.ket_luan_tham_dinh	Silver	ma_giao_dich	Append /	Lưu trữ nhãn phản hồi

			Merge	xác nhận gian lận hoặc hợp lệ từ thẩm định viên ngân hàng.
gold.diem_rui_ro	Gold	ma_giao_dich / Partition: ngay	Append + CDF	Chứa điểm số rủi ro thời gian thực [0..1], rank trong ngày và Top 3 yếu tố giải thích XAI.
gold.theo_doi_drift	Gold	ngay_kiem_tra, ten_bien	Append-Only	Ghi nhận chỉ số PSI và KS phục vụ cảnh báo biến động hành vi và kích hoạt Retrain.

3.3. Thiết kế cơ chế chống rò rỉ dữ liệu (Point-in-Time Join)

Trong quá trình xây dựng tập huấn luyện (training dataset), thách thức lớn nhất là làm sao tính toán các đặc trưng tổng hợp (như số giao dịch trong 1 giờ qua của tài khoản) mà không để rò rỉ các giao dịch xảy ra sau thời điểm giao dịch hiện tại T. Thiết kế Point-in-Time Join tuân thủ nghiêm ngặt nguyên tắc thời gian:

Đối với một giao dịch i của tài khoản A phát sinh tại thời điểm Ti, tập hợp các giao dịch lịch sử dùng để tổng hợp đặc trưng H(A, Ti) được định nghĩa theo công thức toán học sau:

$$H(A, T_i) = \{\ d_j \mid \text{tài_khoản}(d_j) = A \ \wedge\ T_i - \Delta t \leq \text{thời_điểm}(d_j) < T_i\ \}$$

Trong đó: A là mã định danh tài khoản thẻ; Ti là mốc thời gian phát sinh của giao dịch hiện tại đang xét; dj là một giao dịch bất kỳ trong lịch sử; Δt là độ rộng cửa sổ trượt quá khứ (ví dụ 1 giờ, 24 giờ, 7 ngày, 30 ngày).

Ý nghĩa quyết định của dấu ngặt '<': Trong biểu thức 'thời_điểm(dj) < Ti', dấu nhỏ hơn ngặt mang tính sống còn đối với mô hình học máy. Tuyệt đối không được lấy dấu nhỏ hơn hoặc bằng ('<='), vì nếu lấy dấu bằng thì chính bản thân giao dịch i sẽ bị tính gộp vào tập lịch sử của chính nó, gây nên hiện tượng rò rỉ nhãn và rò rỉ dữ liệu tương lai (Data Leakage).

Kỹ thuật này được hiện thực hóa trực tiếp trong PySpark thông qua định nghĩa cấu trúc Window phân vùng và trượt theo thời gian:

```
Window.partitionBy('tai_khoan').orderBy('thoi_diem').rangeBetween(-3600, -1)
```

3.4. Thiết kế cơ chế điều phối 4 cửa sổ Terminal và giao thức IPC đồng bộ

Để phục vụ công tác demo trực quan và bảo vệ đồ án trước Hội đồng, hệ thống thiết kế cơ chế chạy song song 4 cửa sổ Terminal độc lập thông qua script PowerShell `chay.ps1`:

- Cửa sổ 1: Web Flask Server (web/app.py, http://127.0.0.1:5000) lắng nghe các yêu cầu HTTP từ trình duyệt.
- Cửa sổ 2: Spark Streaming Chấm điểm (streaming/cham_diem.py, Spark UI 4040, trigger 2s) chấm điểm ML thời gian thực.

- Cửa sổ 3: Spark Stateful Streaming (streaming/cap_nhat_trang_thai.py, Spark UI 4042, trigger 3s) cập nhật hồ sơ khách hàng Silver.
- Cửa sổ 4: Kafka Producer (data/generator.py) điều khiển phát giao dịch trực tuyến.

Đặc biệt, hệ thống sử dụng giao thức IPC non-blocking kết hợp thư viện `msvcrt.kbhit()` và file cờ tín hiệu `tmp/web_trigger.json`. Khi người dùng bấm nút trên Web, tín hiệu truyền tức thì sang Cửa sổ 4 trong vòng < 0.1 giây để Cửa sổ 4 in các dòng giao dịch trực tiếp lên màn hình console, đồng thời tăng số đợt đồng bộ mà không bao giờ bị trùng lặp giao dịch.

3.5. Thiết kế vòng lặp phản hồi thẩm định Human-In-The-Loop

Mô hình học máy không thể hoạt động đơn độc mà cần sự tương tác phối hợp chặt chẽ với chuyên viên ngân hàng:

- 1. Hàng đợi ưu tiên: Giao dịch có điểm rủi ro cao nhất được đẩy lên đầu danh sách thẩm định trên Web App kèm nhãn màu sắc trực quan.
- 2. Giải trình minh bạch (XAI): Web hiển thị Top 3 yếu tố rủi ro then chốt (Độ lệch chuẩn Z-score số tiền, Đặc trưng ẩn danh PCA V14, Tần suất giao dịch cửa sổ 1 giờ).
- 3. Thao tác thẩm định: Chuyên viên nhấn 'Xác nhận Gian lận' hoặc 'Xác nhận Hợp lệ'. Hệ thống ghi nhận tức thì vào `silver.ket_luan_tham_dinh` với độ trễ < 100ms và đồng bộ lên HDFS.
- 4. Khép kín vòng lặp MLOps: Bảng kết luận thẩm định trở thành nguồn nhãn vàng (Ground Truth) để Airflow định kỳ kiểm tra. Khi tích lũy đủ >= 500 nhãn mới, DAG sẽ tự động kích hoạt tái huấn luyện mô hình mới và tự động chuyển đổi sang phiên bản tiếp theo.

CHƯƠNG 4: HIỆN THỰC HÓA VÀ TRIỂN KHAI HỆ THỐNG

4.1. Môi trường triển khai và hạ tầng cụm phân tán

Hạ tầng hệ thống được hiện thực hóa bằng Docker Compose kết hợp môi trường native trên máy trạm Windows:

Bảng 4.1: Thông số phần cứng, cổng mạng và môi trường triển khai cụm phân tán

Thành phần hệ thống	Công nghệ / Phiên bản	Cấu hình cổng / Tài nguyên	Vai trò và chức năng
HDFS NameNode (namenode1)	Apache Hadoop 3.2.1	Port 9870 (WebUI), 9000 (IPC)	Quản lý siêu dữ liệu tập tin phân tán và điều phối block.
HDFS DataNodes (datanode1..3)	Apache Hadoop 3.2.1	Port 9864, 9865, 9866	Lưu trữ các khối block dữ liệu với hệ số nhân bản replication = 3.
Apache Kafka Broker	Confluent Kafka 2.12	Port 9092 (PLAINTEXT)	Hàng đợi thông điệp phân tán cho các topics `transactions`, `alerts`.
Apache Zookeeper	Zookeeper 3.4	Port 2181	Điều phối cụm Kafka broker và quản lý bầu cử trưởng cụm.
Spark Streaming Engine	Apache Spark 3.5.0	Port 4040, 4042 (UI)	Thực thi chấm điểm vi lô và cập nhật trạng thái hồ sơ người dùng.
Apache Airflow	Airflow 2.8.1 (LocalExecutor)	Port 8080 (Webserver)	Điều phối 4 DAGs tự động hóa giám sát, drift và tái huấn luyện.
Web Application Server	Python 3.11.9 / Flask	Port 5000 (HTTP)	Bảng điều khiển trực quan 8 bước và Bàn làm việc thẩm định viên.

4.2. Hiện thực hóa phân hệ Streaming chấm điểm thời gian thực (Cửa sổ 2)

Phân hệ chấm điểm (`streaming/cham_diem.py`) được thiết kế vận hành với cờ tối ưu hóa cao `--turbo`:

• Trigger chu kỳ vi lô: 2 giây (`processingTime='2 seconds'`).

• Stream-Static Join: Nạp bảng hồ sơ khách hàng `silver.trang_thai_khach_hang` dưới dạng Static DataFrame vào bộ nhớ đệm (broadcast cache), thực hiện phép kết hợp cực nhanh theo `tai_khoan` mà không phải quét lại đĩa.

• Áp dụng PipelineModel: Mô hình GBTClassifier nạp sẵn trích xuất 30 đặc trưng (`V1` đến `V28`, `so_tien`, và `so_giao_dich_1h`), sinh ra cột vector xác suất `probability`.

• Dual-Sink: Ghi song song kết quả vào `bronze.giao_dich` và `gold.diem_rui_ro` với cơ chế kiểm soát checkpoint nghiêm ngặt.

4.3. Hiện thực hóa phân hệ Stateful Streaming cập nhật trạng thái (Cửa sổ 3)

Phân hệ cập nhật hồ sơ khách hàng (`streaming/cap_nhat_trang_thai.py`) chịu trách nhiệm duy trì 'ký ức' hành vi người dùng:

• Cấu trúc trạng thái: Quản lý cửa sổ trượt hành vi trên 4 mốc thời gian: 1 giờ, 24 giờ, 7 ngày và 30 ngày (tính toán `so_giao_dich`, `tong_tien`, `tien_trung_binh`, `do_lech_chuan`).

• Thao tác Delta MERGE INTO: Cứ mỗi 3 giây, các bản ghi biến động trong mẻ vi mô được gom nhóm và áp dụng lệnh MERGE vào `silver.trang_thai_khach_hang` theo khóa `tai_khoan`.

• Quản lý bộ nhớ trạng thái: Tự động giải phóng (timeout eviction) đối với các tài khoản không phát sinh giao dịch mới trong vòng 30 ngày, tránh tràn RAM.

4.4. Hiện thực hóa bộ phát dữ liệu đa chế độ và đồng bộ Web (Cửa sổ 4)

Bộ phát dữ liệu trực tuyến (`data/generator.py`) hỗ trợ 3 chế độ vận hành linh hoạt:

- 1. Chế độ Mặc định: Phát 10 - 20 giao dịch/giây phục vụ kiểm thử chức năng cơ bản.
- 2. Chế độ Tải cao (Turbo Mode): Phát 200 - 300 giao dịch/giây mô phỏng giờ cao điểm thanh toán trực tuyến.
- 3. Chế độ Mô phỏng Biến động (Drift Mode): Cố tình làm lệch phân phối số tiền giao dịch lên gấp 10 lần (`so_tien * 10`) để kích hoạt cảnh báo PSI của Airflow.

Để đồng bộ với giao diện Web mà không làm đóng băng tiến trình, bộ phát sử dụng luồng đọc phím phi phong bế `msvcrt.kbhit()`. Khi nhận được tín hiệu kích hoạt từ Web, bộ phát lập tức xuất các dòng giao dịch mới lên terminal Cửa sổ 4, đồng bộ số đợt nạp.

4.5. Hiện thực hóa giao diện Web điều hành và bàn làm việc thẩm định (Cửa sổ 1)

Ứng dụng Web Flask (`web/app.py`, Port 5000) được thiết kế hiện đại với kiến trúc phân trang tối ưu:

- • Dashboard quy trình 8 bước: Hướng dẫn người dùng thực hiện tuần tự từ nạp dữ liệu, kiểm tra luồng, thẩm định rủi ro, kiểm toán Time Travel đến kích hoạt huấn luyện lại.
- • Phân trang danh sách giao dịch: Giới hạn 5.000 bản ghi mỗi trang hiển thị trên tổng số hàng chục ngàn giao dịch trong hồ Lakehouse, đảm bảo giao diện phản hồi cực nhanh dưới 50ms.
- • Bàn làm việc thẩm định Human-In-The-Loop: Chuyên viên chọn giao dịch, xem điểm rủi ro, xem Top 3 yếu tố cảnh báo, và nhấn nút 'Xác nhận Gian lận' hoặc 'Xác nhận Hợp lệ'.

4.6. Hiện thực hóa các Pipeline điều phối Apache Airflow

Hệ thống xây dựng 4 DAGs tự động hóa vận hành toàn diện:

- DAG 1: gl_giam_sat_streaming: Chạy mỗi 5 phút, kiểm tra tính sống còn của 2 tiến trình Spark Streaming và Kafka Broker.

- DAG 2: gl_kiem_tra_drift: Chạy mỗi 60 phút, tính toán chỉ số PSI và KS-test trên bảng `gold.diem_rui_ro` so với tập cơ sở. Khi PSI > 0.25, DAG tự động gửi cảnh báo và kích hoạt tái huấn luyện.
- DAG 3: gl_huan_luyen_lai: Kiểm tra số lượng nhãn mới trong `silver.ket_luan_tham_dinh`. Khi đạt đủ >= 500 nhãn, DAG thực hiện Point-in-Time Join, huấn luyện mô hình ứng viên mới và hoán đổi mô hình tự động.
- DAG 4: gl_bao_tri: Chạy định kỳ lúc 2:00 AM Chủ Nhật, thực hiện lệnh `OPTIMIZE Z-ORDER BY (tai_khoan, thoi_diem)` và `VACUUM RETAIN 168 HOURS` để dọn dẹp các tệp tin rác.

CHƯƠNG 5: THỬ NGHIỆM, ĐÁNH GIÁ VÀ NGHIỆM THU HỆ THỐNG

5.1. Kịch bản thử nghiệm áp lực cao (Turbo Mode Stress Test)

Kịch bản thử nghiệm áp lực cao được tiến hành liên tục trong 30 phút trên cụm phân tán:

- Tốc độ phát sinh giao dịch: 300 giao dịch/giây (vượt 3 lần công suất đỉnh giờ cao điểm của ngân hàng thương mại quy mô vừa).
- Tỷ lệ giao dịch lệch thứ tự (out-of-order): 15% gói tin bị trễ từ 5 đến 30 giây.
- Tỷ lệ bản ghi lỗi cố ý: 2% giao dịch bị khuyết thiếu khóa chính hoặc số tiền âm để kiểm thử phân hệ Quarantine.

5.2. Đánh giá đối sánh hiệu năng các mô hình học máy

Nhóm đã tiến hành thực nghiệm đối sánh 3 thuật toán học máy kết hợp 2 chiến lược xử lý mất cân bằng trên tập kiểm thử độc lập (Test Set gồm 56.962 giao dịch với 98 ca gian lận):

Bảng 5.1: Bảng đối sánh hiệu năng 3 thuật toán x 2 chiến lược mất cân bằng trên tập Test

Thuật toán	Xử lý mất cân bằng	Precision@200	PR-AUC	ROC-AUC	Ngưỡng	Trạng thái
GBTClassifier	WeightCol (~577.0)	0.4800	0.8814	0.9759	0.50	Được chọn đưa vào Production
GBTClassifier	Undersampling (1:20)	0.3800	0.8120	0.9610	0.45	Dự phòng (Precision@200 giảm)
Random Forest	WeightCol (~577.0)	0.4200	0.8350	0.9680	0.50	Đạt chuẩn nhưng kém hơn GBT
Random Forest	Undersampling (1:20)	0.3500	0.7890	0.9520	0.40	Đạt ngưỡng tối thiểu
Logistic Regression	WeightCol (~577.0)	0.2200	0.6420	0.9100	0.50	Không đạt chuẩn Precision@200
Logistic Regression	Undersampling (1:20)	0.1900	0.5810	0.8950	0.40	Không đạt chuẩn ngân hàng

Phân tích kết quả thực nghiệm: Mô hình GBTClassifier kết hợp trọng số lớp WeightCol đạt kết quả vượt trội với Precision@200 = 0.4800 (trong 200 cảnh báo có tới 96 ca gian lận thật) và PR-AUC = 0.8814. Kết quả này vượt xa chỉ tiêu cam kết ban đầu (Precision@200 >=

0.35 và PR-AUC >= 0.75), chứng minh tính ưu việt của thuật toán cây tăng cường độ dốc trong việc phân tách dữ liệu gian lận phi tuyến.

5.3. Đánh giá độ trễ xử lý đầu cuối (End-to-End Latency)

Độ trễ đầu cuối được đo lường từ thời điểm gói tin giao dịch được đẩy vào Kafka Producer đến thời điểm kết quả chấm điểm rủi ro xuất hiện trong bảng Delta Lake `gold.diem_rui_ro`:

Bảng 5.2: Đo lường độ trễ xử lý End-to-End tại các mốc phân vị (P50, P95, P99)

Mốc phân vị thống kê	Thời gian đo lường thực tế	Cam kết chất lượng (SLA)	Đánh giá kết quả
Phân vị 50% (P50 - Trung vị)	820.5 ms (0.82 giây)	< 2.000 ms (2.0 giây)	ĐẠT (Vượt cam kết 59%)
Phân vị 95% (P95)	1.285.4 ms (1.28 giây)	< 5.000 ms (5.0 giây)	ĐẠT (Vượt cam kết 74%)
Phân vị 99% (P99 - Đỉnh tải)	1.342.0 ms (1.34 giây)	< 5.000 ms (5.0 giây)	ĐẠT (Vượt cam kết 73%)

5.4. Kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống

Bảng 5.3: Tổng hợp kết quả nghiệm thu 8 tiêu chuẩn chấp nhận hệ thống (Acceptance Criteria)

Tiêu chuẩn nghiệm thu	Yêu cầu kỹ thuật & nghiệp vụ	Kết quả thực nghiệm	Kết luận
AC1: Hạ tầng Medallion	5 bảng Delta Lake phân tầng, lưu trữ trên HDFS với replication=3.	100% kiểm tra hợp lệ, HDFS phân tán ổn định.	CHẤP NHẬN
AC2: Luồng Streaming	Hai luồng Spark Structured Streaming vi lô 2-3s, Dual-Sink.	Xử lý trơn tru, không nghẽn luồng.	CHẤP NHẬN
AC3: Điểm rủi ro & XAI	Chấm điểm xác suất [0..1] kèm Top 3 giải thích rủi ro minh bạch.	Đầy đủ cột xác suất và giải trình XAI.	CHẤP NHẬN
AC4: Hiệu năng mô hình	Precision@200 >= 0.35; PR-AUC >= 0.75 trên tập Test.	Đạt Precision@200 = 0.4800, PR-AUC = 0.8814.	CHẤP NHẬN
AC5: Cam kết độ trễ SLA	P95 End-to-End Latency < 5.0s (kỳ vọng P50 < 2.0s).	Thực tế P50 = 820.5ms, P95 = 1.285.4ms.	CHẤP NHẬN
AC6: Vòng lặp	Bàn làm việc thẩm định	Độ trễ ghi nhãn < 100ms,	CHẤP NHẬN

phản hồi	Human-In-The-Loop ghi nhận nhãn vàng.	đồng bộ tức thì.	
AC7: Tự động hóa MLOps	4 DAGs Airflow giám sát, kiểm tra drift PSI và kích hoạt retrain.	Hoạt động tự động, kích hoạt đúng ngưỡng.	CHẤP NHẬN

5.5. Phân tích các ca xử lý lỗi kỹ thuật thực tế trong quá trình phát triển

Trong quá trình phát triển và kiểm thử hệ thống, nhóm đã đối mặt và giải quyết thành công 3 sự cố kỹ thuật phức tạp:

1. Sự cố xung đột chỉ số phiên bản Delta Lake ngắt quãng: Do quá trình dọn dẹp thư mục commit log trước đó, các file commit bị mất tính liên tục (ví dụ có file 0 và file 2 nhưng thiếu file 1). Động cơ Delta Engine phát sinh ngoại lệ `DeltaNonContiguousVersionsException`. Nhóm đã khắc phục bằng cách bổ sung cơ chế kiểm tra `last_ver_num + 1` trong `DeltaFastEngine`, tự động tạo commit bù đắp mà không làm mất tính toàn vẹn dữ liệu.

2. Sự cố xung đột cổng dịch vụ Spark UI và lỗi JVM Java 17 module reflection: Khi chạy song song Cửa sổ 2 và Cửa sổ 3, cả hai tiến trình cùng cố gắng liên kết cổng mặc định 4040, đồng thời JVM Java 17 chặn các truy cập nội bộ `sun.nio.ch.DirectBuffer` gây lỗi `BlockManagerId.executorId() NPE`. Nhóm đã giải quyết triệt để bằng cách cấu hình cổng động (`spark.ui.port=4042`), tăng thời gian chờ `spark.network.timeout=300s` và bổ sung các cờ JVM `--add-opens` trong lệnh khởi tạo.

3. Sự cố đồng bộ bất đồng bộ giữa Web và Terminal Cửa sổ 4: Khi người dùng bấm nút nạp giao dịch trên Web, Cửa sổ 4 bị phong bế do hàm `input()` chờ phím Enter. Nhóm đã tái cấu trúc cơ chế IPC bằng cách sử dụng file cờ `tmp/web_trigger.json` kết hợp hàm kiểm tra phi phong bế `msvcrt.kbhit()`, cho phép nhận lệnh tức thời từ Web trong vòng < 0.1 giây.

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Các kết quả và đóng góp chính của đề tài

Đề tài 'Xây dựng Hệ thống Phát hiện Gian lận Thẻ Tín dụng Real Time trên Nền tảng Big Data Lakehouse' đã hoàn thành xuất sắc toàn bộ các mục tiêu đặt ra ban đầu, với 5 đóng góp chính:

- 1. Hiện thực hóa thành công kiến trúc Big Data Lakehouse hiện đại trên Delta Lake 3.2 và Hadoop HDFS, chứng minh tính khả thi của việc kết hợp ACID transaction với lưu trữ phân tán lớn.
- 2. Tối ưu hóa chu trình xử lý luồng dữ liệu thời gian thực đạt độ trễ P50 = 820.5ms và P95 = 1.285ms, đáp ứng tiêu chuẩn khắt khe của hệ thống thanh toán ngân hàng quốc tế.
- 3. Xây dựng mô hình học máy GBTClassifier đạt hiệu năng cao trên dữ liệu mất cân bằng cực đoan (Precision@200 = 0.4800, PR-AUC = 0.8814) và giải quyết triệt để bài toán rò rỉ dữ liệu bằng Point-in-Time Join.
- 4. Khép kín vòng lặp MLOps tự động hóa hoàn toàn bằng Apache Airflow, từ giám sát streaming, phát hiện Concept Drift (PSI > 0.25) đến tái huấn luyện tự động.
- 5. Cung cấp bộ giải pháp công nghệ trực quan với kịch bản điều phối 4 cửa sổ đồng bộ và giao diện Web điều hành tích hợp bàn làm việc thẩm định Human-In-The-Loop.

6.2. Hạn chế còn tồn tại

Bên cạnh những kết quả nổi bật đã đạt được, hệ thống vẫn còn một số điểm hạn chế do giới hạn về thời gian và tài nguyên phần cứng thử nghiệm:

- • Môi trường triển khai máy trạm Windows: Hệ thống Spark Streaming và Docker chạy trên nền tảng Windows native đòi hỏi nhiều tinh chỉnh tham số môi trường và phụ thuộc vào tệp winutils.
- • Khả năng chịu lỗi phần cứng hạn chế: Các dịch vụ Docker được cấu hình trên cùng một máy chủ vật lý, chưa mô phỏng được các trường hợp đứt cáp mạng giữa các rack (Network Partitioning).
- • Phương pháp giải trình mô hình (XAI): Hệ thống mới trích xuất các trọng số đặc trưng tĩnh và Z-score, chưa tích hợp thuật toán SHAP hay LIME động trong thời gian thực.

6.3. Định hướng phát triển trong tương lai

Để nâng cấp hệ thống lên quy mô sản phẩm cấp thương mại (Production-grade), các hướng nghiên cứu tiếp theo bao gồm:

- 1. Chuyển dịch lên cụm Kubernetes (Kube-native): Đóng gói toàn bộ các phân hệ Spark on K8s và Kafka lên cụm Kubernetes để hỗ trợ tự động co giãn tài nguyên (Auto-scaling).
- 2. Ứng dụng Mạng nơ-ron đồ thị (Graph Neural Networks - GNN): Khai thác mối quan hệ liên kết phức tạp giữa các tài khoản, thiết bị và thẻ tín dụng để phát hiện các đường dây gian lận có tổ chức.
- 3. Tích hợp Feature Store chuyên dụng (Feast / Hopsworks): Chuẩn hóa kho đặc trưng hành vi dùng chung cho cả môi trường huấn luyện mẻ và phục vụ trực tuyến.

TÀI LIỆU THAM KHẢO

[1] Armbrust, M., Das, T., Sun, L., Yavuz, B., Zhu, S., Murthy, M., ... & Zaharia, M. (2020). 'Delta Lake: High-Performance ACID Table Storage over Cloud Object Stores'. Proceedings of the VLDB Endowment, 13(12), 3411-3424.

[2] Kreps, J., Narkhede, N., & Rao, J. (2011). 'Kafka: A Distributed Messaging System for Log Processing'. Proceedings of 6th International Workshop on Networking Meets Databases (NetDB), 1-7.

[3] Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., ... & Stoica, I. (2016). 'Apache Spark: A Unified Engine for Big Data Processing'. Communications of the ACM, 59(11), 56-65.

[4] Dal Pozzolo, A., Boracchi, G., Caelen, O., Alippi, C., & Bontempi, G. (2018). 'Credit Card Fraud Detection: A Realistic Modeling and a Novel Learning Strategy'. IEEE Transactions on Neural Networks and Learning Systems, 29(8), 3784-3797.

[5] Dal Pozzolo, A., Caelen, O., Johnson, R. A., & Bontempi, G. (2015). 'Calibrating Probability with Undersampling for Unbalanced Classification'. IEEE Symposium Series on Computational Intelligence (SSCI), 159-166.

[6] Carcillo, F., Le Borgne, Y. A., Caelen, O., & Bontempi, G. (2018). 'Streaming Active Learning Strategies for Real-Life Credit Card Fraud Detection: Assessment and Visualization'. International Journal of Data Science and Analytics, 5(4), 285-300.

[7] Yurdakul, B. (2018). 'Statistical Properties of the Population Stability Index'. Doctoral Dissertation, Western Michigan University.

[8] Apache Software Foundation. (2024). 'Apache Spark Structured Streaming Programming Guide'. Official Documentation, spark.apache.org.

[9] Delta Lake Project. (2024). 'Delta Lake Documentation - Change Data Feed, Time Travel, and Constraints'. docs.delta.io.

[10] Apache Airflow. (2024). 'Airflow Architecture and Best Practices for Data Pipelines'. airflow.apache.org.

PHỤ LỤC: HƯỚNG DẪN CÀI ĐẶT VÀ VẬN HÀNH HỆ THỐNG

Phụ lục A: Hướng dẫn cấu hình Docker Compose và kiểm tra hạ tầng

Hạ tầng cụm phân tán được khởi động thông qua Docker Compose tại thư mục gốc của dự án:

```
docker compose up -d
docker compose ps
```

Kiểm tra giao diện WebHDFS NameNode tại địa chỉ: http://localhost:9870 để xác nhận đủ 3 DataNodes (datanode1, datanode2, datanode3) đang hoạt động bình thường ở trạng thái 'In service'.

Phụ lục B: Hướng dẫn khởi động hệ thống 4 cửa sổ đồng bộ bằng chay.ps1

Hệ thống được tự động hóa khởi động bằng một lệnh duy nhất trong PowerShell:

```
powershell -ExecutionPolicy Bypass -File .\chay.ps1
```

Lệnh trên sẽ tự động kích hoạt và sắp xếp 4 cửa sổ Terminal nổi tương ứng trên màn hình:

- Cửa sổ 1: Web Flask Server điều hành (http://127.0.0.1:5000)
- Cửa sổ 2: Spark Streaming Chấm điểm rủi ro (cham_diem.py --turbo)
- Cửa sổ 3: Spark Stateful Streaming cập nhật trạng thái hồ sơ (cap_nhat_trang_thai.py --turbo)
- Cửa sổ 4: Kafka Producer Phát giao dịch trực tuyến (generator.py)

Phụ lục C: Hướng dẫn làm sạch dữ liệu và chạy bộ kiểm thử nghiệm thu

Để làm sạch toàn bộ dữ liệu giao dịch về trạng thái ban đầu v0 và khôi phục mô hình gốc model_v1:

```
.\venv311\Scripts\python.exe reset_he_thong.py
```

Để thực thi bộ kiểm thử nghiệm thu tự động kiểm chứng 8 tiêu chuẩn chấp nhận hệ thống:

```
.\venv311\Scripts\python.exe tests/test_acceptance.py
```